

Detección de Objetos y Control de Brazo Robótico mediante Simulación en Processing (Equipo 1)

H. D. Castillo Ovando*, J. J. Jauregui Barba†, R. J. Hillario Cruz‡,
R. A. De Los Rios Moreno§, J. A. Garcia Gonzales¶, V. A. Meneses Garza||, J. A. Palafox Salinas**
Instituto Tecnológico y de Estudios Superiores de Monterrey, Monterrey, México
*A00836025, †A00836722, ‡A00835860, §A01198515,
¶A01369587, ||A01384002, **A00835705
*A00836025@tec.mx, †A00836722@tec.mx, ‡A00835860@tec.mx, §A01198515@tec.mx,
¶A01369587@tec.mx, ||A01384002@tec.mx, **A00835705@tec.mx

Resumen—Este documento presenta un estudio integral en el que se combinan técnicas de procesamiento de imágenes y visión por computador para la detección y registro de objetos, junto con la simulación de un brazo robótico en Processing. Se detalla el uso de filtros de detección (como los operadores Sobel, Canny y SUSAN) para extraer contornos y puntos de interés, y se describe cómo la estimación de la pose del objeto (posición y orientación) se traduce en órdenes de movimiento mediante algoritmos de cinemática inversa. Además, se diseña una estrategia interactiva en la que el usuario selecciona un punto en la imagen para que, en la simulación, el brazo robótico lleve el objeto a la ubicación deseada. Se discuten fundamentos teóricos, implementaciones prácticas y posibles líneas de mejora para aplicaciones en sistemas de control en tiempo real y robótica colaborativa.

Index Terms—Procesamiento de imágenes, visión por computador, detección de objetos, cinemática inversa, brazo robótico, simulación en Processing.

I. INTRODUCCIÓN

El avance en visión por computador y en la integración de sistemas de control ha permitido la implementación de soluciones complejas en robótica. En contextos donde se requieren tiempos de respuesta muy cortos, como en el control de brazos robóticos, es importante disponer de algoritmos que procesen imágenes en tiempo real y determinen la pose del objeto de interés. La tesis de Atoche [1] presentó una solución basada en FPGA para acelerar el procesamiento de imágenes y el registro de objetos en tiempo real.

Este reporte se enfoca en dos áreas fundamentales:

1. **Detección de Objetos:** Se profundiza en el uso de filtros para la extracción de contornos y características relevantes, permitiendo identificar la orientación del objeto a partir de una imagen.
2. **Simulación del Brazo Robótico en Processing:** Se describe cómo la información obtenida se utiliza para generar, mediante algoritmos de cinemática inversa, una trayectoria de movimiento que permita a un brazo robótico simulado posicionar el objeto en un punto específico del espacio de trabajo interactivo.

La estructura del documento abarca desde los fundamentos del procesamiento de imágenes hasta la integración completa del sistema y las líneas futuras de investigación.

II. FUNDAMENTOS DEL PROCESAMIENTO DE IMÁGENES Y VISIÓN POR COMPUTADOR

II-A. Clasificación y Niveles de Procesamiento

Para transformar una imagen digital en información útil para la robótica, se distinguen tres niveles de procesamiento:

- **Procesamiento de Bajo Nivel:** Se manipulan directamente los píxeles. Incluye el uso de filtros espaciales (pasa bajos y pasa altos), técnicas de mejora de imagen y operadores de detección de bordes (Sobel, Laplaciano, Canny y SUSAN).
- **Procesamiento de Medio Nivel:** Se ocupa de la segmentación y extracción de características. La segmentación separa el objeto del fondo y la extracción de contornos y puntos clave genera un mapa de información.
- **Procesamiento de Alto Nivel:** Con la información extraída se interpreta y modela la escena. Se realiza el registro o "matching" entre la imagen 2D y un modelo 3D, obteniendo la pose del objeto.

Pregunta: ¿Qué tipo de procesamiento se debe llevar a cabo?

Respuesta: Se debe aplicar un procesamiento jerárquico: primero técnicas de bajo nivel (filtrado y detección de bordes), luego medio nivel (segmentación y extracción de características) y finalmente alto nivel (registro del objeto y estimación de su pose).

II-B. Filtros de Detección de Contornos y Extracción de Características

La detección de contornos es crucial para identificar los límites del objeto. Entre los operadores más utilizados se encuentran:

- **Operador Sobel:** Calcula derivadas parciales en direcciones x e y , resaltando zonas de cambio brusco.
- **Operador Laplaciano:** Evalúa la segunda derivada, detectando bordes con cambios rápidos.
- **Operador Canny:** Método sofisticado que incluye filtrado gaussiano, cálculo de gradiente, supresión de no máximos y umbralización doble.

- **Filtro SUSAN:** Se basa en el concepto de "área de similitud.^{en} la vecindad de cada píxel, permitiendo detectar bordes y esquinas de manera robusta ante el ruido.

Una vez detectados los contornos, se extraen puntos característicos mediante técnicas de análisis de momentos y otros métodos estadísticos, lo que permite obtener la orientación del objeto.

II-C. Registro y Estimación de la Pose del Objeto

El registro implica alinear los datos extraídos de la imagen 2D con un modelo 3D predefinido. Esto se realiza mediante:

- **Apareamiento de Puntos:** Algoritmos iterativos (como los métodos de Wunsch o Lowe) que buscan correspondencias entre la imagen y el modelo.
- **Cinemática del Objeto:** Cálculo de traslación y rotación para ajustar el modelo a la imagen, utilizando técnicas como la descomposición en valores singulares (SVD).

La estimación precisa de la pose es fundamental para trasladar la información al entorno de simulación y determinar el movimiento del brazo robótico.

III. ESTRATEGIA DE INTEGRACIÓN: DE LA IMAGEN AL MOVIMIENTO DEL BRAZO ROBÓTICO

III-A. Análisis de Imagen y Conversión a Información de Movimiento

El proceso inicia con la captura de una imagen a través de una cámara calibrada, lo que permite conocer los parámetros intrínsecos y extrínsecos. Los pasos son:

1. **Adquisición y Preprocesamiento:** Captura y mejora de la imagen mediante filtrado y ajuste de contraste.
2. **Aplicación del Filtro de Detección:** Uso del operador seleccionado (SUSAN, Canny, etc.) para extraer contornos.
3. **Extracción de Características y Estimación de la Pose:** Identificación de puntos clave para determinar la orientación y posición del objeto.

Pregunta: ¿Cómo se logra que, mediante la simulación del brazo robótico, el objeto se coloque en el ángulo detectado?

Respuesta: La pose estimada se traduce en datos de posición y orientación. Mediante algoritmos de cinemática inversa se calculan los ángulos de las articulaciones del brazo que permiten replicar la orientación detectada en la imagen.

III-B. Mapeo de Coordenadas y Selección del Punto de Destino

Una vez estimada la pose, se permite la interacción del usuario para definir un punto de destino:

- **Interfaz de Usuario en Processing:** Se desarrolla una interfaz gráfica para que el usuario seleccione el punto destino mediante clics.
- **Transformación de Coordenadas:** Los parámetros de calibración permiten mapear las coordenadas de la imagen al espacio de trabajo.
- **Definición de la Trayectoria:** Con la posición actual y el punto destino se calcula la trayectoria mediante métodos

de interpolación (lineal o splines) para garantizar un movimiento suave.

Pregunta: Diseña una estrategia para que yo le diga un punto en la imagen y que un robot mueva el objeto.

Respuesta: La estrategia consiste en convertir el punto indicado en la imagen a coordenadas del espacio de trabajo y, a partir de la posición inicial y final, generar una trayectoria mediante cinemática inversa que mueva el objeto de forma suave y precisa.

III-C. Generación del Movimiento: Cinemática Inversa y Control del Brazo

El movimiento del brazo se divide en dos fases:

1. **Orientación del Objeto:** Se ajusta la orientación del objeto utilizando cinemática inversa para calcular los ángulos de las articulaciones.
2. **Traslación al Punto Destino:** Se genera una trayectoria interpolada desde la posición inicial hasta el punto destino, asegurando una transición suave.

La integración de estas fases se implementa en Processing, donde cada frame se actualizan los parámetros del brazo robótico para simular el movimiento en tiempo real.

IV. IMPLEMENTACIÓN PRÁCTICA EN PROCESSING

IV-A. Arquitectura del Sistema en Processing

La solución se estructura en módulos interconectados:

- **Módulo de Captura y Preprocesamiento:** Carga la imagen y aplica filtros para mejorarla y extraer contornos.
- **Módulo de Análisis y Estimación de Pose:** Procesa la imagen filtrada para calcular la orientación y posición del objeto.
- **Módulo de Interfaz e Interacción:** Permite al usuario seleccionar el punto destino mediante eventos de mouse.
- **Módulo de Control y Cinemática Inversa:** Calcula los ángulos de las articulaciones del brazo y genera la trayectoria de movimiento.



Figura 1: Demostración de la simulación inicial

IV-B. Pseudocódigo Mejorado

Algorithm 1 Algoritmo de Control del Brazo Robótico

```
1: Inicializar brazo robótico
2: Cargar imagen de entrada
3: Aplicar filtro de detección (Sobel/Canny/SUSAN)
4: Detectar contornos y características
5: Calcular pose del objeto (posición, orientación)
6: while simulación activa do
7:   if usuario ha seleccionado punto destino then
8:     Calcular trayectoria usando cinemática inversa
9:     Mover brazo a posición objetivo
10:  end if
11:  Actualizar visualización
12: end while
```

IV-C. Ejemplo de Código en Processing

```
1 // Clase principal del sistema
2 import processing.video.*;
3
4 PImage imgOriginal;
5 PVector puntoDestino;
6 BrazoRobot brazo;
7 Capture cam;
8
9 void setup() {
10   size(800, 600);
11   // Inicializar cámara (si se usa captura en vivo)
12   String[] cameras = Capture.list();
13   if (cameras.length > 0) {
14     cam = new Capture(this, cameras[0]);
15     cam.start();
16   } else {
17     imgOriginal = loadImage("imagen_objeto.jpg");
18   }
19
20   brazo = new BrazoRobot();
21   puntoDestino = new PVector(width/2, height/2);
22
23   println("Sistema de control de brazo robótico
24     inicializado");
25 }
26
27 void draw() {
28   background(255);
29
30   // Captura o carga de imagen
31   if (cam != null && cam.available()) {
32     cam.read();
33     imgOriginal = cam.get();
34   }
35
36   // Procesamiento de imagen
37   PImage imgProcesada = aplicarFiltroDeteccion(
38     imgOriginal);
39   float anguloObjeto = calcularAnguloObjeto(
40     imgProcesada);
41
42   // Mostrar imagen procesada
43   image(imgProcesada, 0, 0);
44
45   // Mostrar información
46   fill(0);
47   textSize(16);
48   text("ángulo del objeto: " + nf(anguloObjeto, 1,
49     2) + " °", 10, 20);
```

```
46   text("Posición destino: (" + nf(puntoDestino.x,
47     1, 1) +
48     ", " + nf(puntoDestino.y, 1, 1) + ")", 10,
49     40);
50
51   // Actualizar brazo
52   brazo.actualizar(anguloObjeto, puntoDestino);
53   brazo.dibujar();
54
55   // Visualización del punto destino
56   fill(255, 0, 0, 150);
57   ellipse(puntoDestino.x, puntoDestino.y, 20, 20);
58   fill(255, 0, 0);
59   ellipse(puntoDestino.x, puntoDestino.y, 5, 5);
60 }
61
62 void mouseClicked() {
63   if (mouseX >= 0 && mouseX < width && mouseY >= 0
64     && mouseY < height) {
65     println("Nuevo destino seleccionado: (" + mouseX
66       + ", " + mouseY + ")");
67     puntoDestino = new PVector(mouseX, mouseY);
68   }
69 }
70
71 PImage aplicarFiltroDeteccion(PImage img) {
72   // Implementación del filtro seleccionado
73   PImage result = createImage(img.width, img.height,
74     RGB);
75   // ... código del filtro ...
76   return result;
77 }
78
79 float calcularAnguloObjeto(PImage img) {
80   // Cálculo de la orientación del objeto
81   float angulo = 0;
82   // ... implementación del cálculo ...
83   return angulo;
84 }
```

Listing 1: Implementación principal en Processing

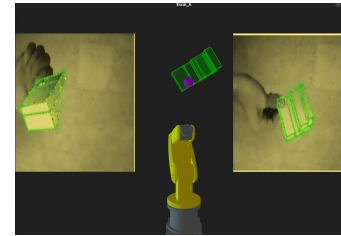


Figura 2: Demostración de la simulación durante el proceso

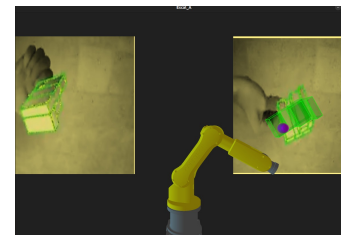


Figura 3: Demostración de la simulación final

IV-D. Detalles Técnicos y Consideraciones de Integración

Para un funcionamiento óptimo, se deben considerar:

- **Calibración Precisa:** Utilizar parámetros de calibración para transformar correctamente las coordenadas.
- **Tasa de Actualización:** Asegurar una actualización de al menos 30 FPS para movimientos fluidos.
- **Interpolación Suave:** Aplicar funciones de interpolación para evitar movimientos abruptos.
- **Validación Experimental:** Realizar pruebas con diferentes escenarios para ajustar los parámetros del sistema.

- Integrar un módulo de aprendizaje automático para mejorar la detección.
- Migrar a una arquitectura multihilo para mejor rendimiento.

El material complementario se encuentra disponible en el siguiente enlace de Google Drive:

https://drive.google.com/drive/folders/165j8atCNZQJDs7ynGBv_tfDj8Eky_fx?usp=sharing

V. CONCLUSIONES

El sistema integrado descrito en este reporte permite transformar datos de imágenes en órdenes de movimiento para un brazo robótico. Se concluye que:

- La integración multinivel (bajo, medio y alto nivel) es esencial para una detección robusta y precisa.
- Los algoritmos de cinemática inversa permiten calcular de manera eficiente los movimientos necesarios.
- La incorporación de una interfaz interactiva en Processing enriquece la experiencia y abre posibilidades para aplicaciones en tiempo real.

VI. REFERENCIAS

REFERENCIAS

- [1] J. R. Atoche, *Procesamiento de Imágenes y Registro de Objetos en Tiempo Real con FPGA*, 2006.
- [2] R. C. González and R. E. Woods, *Digital Image Processing*, 2nd ed. Prentice Hall, 1992.
- [3] J. Canny, "A Computational Approach to Edge Detection," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. PAMI-8, no. 6, pp. 679–698, Nov. 1986.
- [4] Processing Foundation, *Processing Reference*, <https://processing.org/reference>, 2023.
- [5] Bradski, G. and Kaehler, A., *Learning OpenCV: Computer Vision with the OpenCV Library*, O'Reilly Media, 2008.

COMENTARIOS PERSONALES

- **Experiencia con Processing:** Fue desafiante integrar los conceptos de visión por computadora con la simulación física del brazo robótico. Processing resultó ser una herramienta adecuada para prototipado rápido pero encontramos limitaciones en el rendimiento para procesamiento de imágenes complejas.
- **Dificultades encontradas:** La sincronización entre la detección de objetos y el movimiento del brazo requirió ajustes finos en los parámetros de tiempo. También tuvimos problemas con la calibración inicial del sistema de coordenadas.
- **Lecciones aprendidas:**
 - La importancia de la calibración precisa entre el espacio de la imagen y el espacio de trabajo del robot.
 - Necesidad de optimizar los algoritmos de procesamiento de imágenes para tiempo real.
 - Valor de implementar un sistema de logging robusto para depuración.
- **Mejoras identificadas:**
 - Implementar un sistema de colisiones más robusto.
 - Añadir feedback visual más detallado para el usuario.